

CSC-341 DevOps and CC foundation

Instructor: Muhammad Ali

Exam: Fall 2025 Mid Term

Marks: 25

Date: 15-11-25

Time: 60min

Name:

ID:

Instructions:

- 1 Answer all questions.
- 2 Write your answers clearly and provide any necessary explanations, commands, or code snippets as requested.
- 3 Where code or command snippets are requested, ensure they are correctly formatted.
- 4 You have 1 hour to complete the paper. Try to manage your time effectively, and spend about 10-12 minutes per question.
- 5 This is an individual assessment, and you should complete it independently. Any form of academic dishonesty will result in penalties.
- 6 You may not use any electronic devices (except the computer provided for the exam) to access any online resources, including the internet, textbooks, or notes.
- 7 If a question asks for a process or steps, ensure that you follow a clear and logical order in your response.

Course Learning Outcomes (CLOs):

- CLO1: Understand and explain the core principles, tools, and practices of DevOps and cloud computing, including key concepts like IaaS, PaaS, SaaS, and virtualization technologies.
- CLO2: Apply cloud computing platforms (e.g., AWS, Azure, GCP) and tools (e.g., Docker, Kubernetes) to deploy and manage applications and infrastructure.
- CLO3: Develop/Construct CI/CD pipelines using DevOps tools and cloud services, focusing on automation, containerization, and orchestration.
- CLO4: Evaluate and integrate security practices into the DevOps lifecycle, ensuring secure deployments and maintaining best practices for cloud infrastructure and applications

Section A: Git

5

1. You are working on a project where multiple developers are collaborating. You want to work on a new feature without affecting the main codebase. How would you create a new branch for this feature?

[1] CLO2

Name: _____

ID: _____

2. After several commits on your branch, you realize that your changes should be integrated into the main branch. Describe how you would merge your feature branch into the main branch using Git? [1] CLO2

3. You are asked to roll back the codebase to a specific commit before an unwanted change was made. How would you checkout to that specific commit using Git? [1] CLO2

4. You have been working on a project for a while, and now you want to push your local commits to a remote GitHub repository. How would you do that? [1] CLO1, CLO2

5. After forking a repository, you want to contribute to the original repository by submitting your changes. What is the process? [1] CLO2

Section B: Containerization

15

- 1 You are tasked with deploying a web application in a containerized environment. The application requires an external database (such as MySQL) and a Redis cache to be running alongside it. The containers should be networked such that the application can talk to both services. Describe the process, including the Docker commands and configurations needed, to create a containerized setup with network isolation and proper inter-container communication.

Default Ports are:

Flask web Application : 5000

MySQL: 3306

Redis: 6379

[5] CLO2, CLO3

Name: _____

ID: _____

- 2 You are running multiple Docker containers on the same host, and one of the containers needs access to persistent data, but the data should not be lost if the container is deleted. Additionally, you need to ensure that multiple containers can write to this persistent data simultaneously. Describe how you would achieve this? [5] CLO2

Name: _____

ID: _____

- 3 The web application you're deploying requires running multiple containers with different environments (e.g., development, staging, production) and different configurations for each environment. How would you ensure that the correct container setups are deployed in each environment? Explain your answer through example.

[5] CLO2, CLO3

Section C: CI/CD

[5]

You are reviewing a GitHub Actions workflow that has been set up to automate the process of building a Docker image and pushing it to Docker Hub. The workflow is triggered by a push to the master branch. However, after reviewing the workflow file, you notice several issues and areas for improvement. The goal is to ensure that the workflow is both efficient and secure.

CLO3

name: Build and Push Docker Image to Docker Hub

on:

push:

branches:

- master

jobs:

build-and-push:

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v3

- name: Set up Docker Buildx

uses: docker/setup-buildx-action@v2

with:

Name: _____

ID: _____

```
version: latest

- name: Log in to Docker Hub
  uses: docker/login-action@v2
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

- name: Build Docker image
  run: |
    docker build -t username/myapp:latest .

- name: Push Docker image to Docker Hub
  run: |
    docker push username/myapp:latest
```

Issues:

Upon review, you find the following potential issues in the workflow:

- The workflow is triggered only when a commit is pushed to the master branch. However, your team also uses a develop branch for testing new features before merging them into master.
- The workflow tags the Docker image with the latest tag every time. This approach may cause difficulties in tracking which version of the image was built from which commit.

Question 1: You are asked to update the workflow so that it triggers on both master and develop branches. How would you modify the on section of the workflow file? [2] CLO3

Question 2: The current workflow tags the Docker image as latest, but the team wants to include the commit hash in the image tag to ensure that each image is uniquely identifiable. How would you modify the docker build command to use the commit SHA (\${{ github.sha }}) as part of the image tag?

[3] CLO2, CLO3

Name: _____

ID: _____

git init git init my-project git clone <url> git clone <url> my-folder git clone --branch <branch> <url> git status git status -sb git ls-files git diff git diff --staged git diff HEAD git add . git commit -m "Message" git status --ignored git rm --cached file.txt git branch <name> git checkout <name> git switch <name> git checkout main git merge feature-branch git log git show <commit> git stash git remote -v git remote add origin <url> git fetch git fetch origin main git pull git pull --rebase git push git push -u origin main git push origin <branch> git restore file.txt git restore . git revert <commit> git revert HEAD git reset --soft <commit> git reset --mixed <commit> git reset --hard <commit> git reset --soft HEAD~1 git reset --hard HEAD~1 git checkout feature git rebase main git rebase --abort git rebase --continue git rebase -i HEAD~5 git push --force-with-lease	docker info docker build -t <image> . docker build -t <image>:<tag> . docker build -f <Dockerfile> -t <image> . docker tag <image>:<tag> <repo>/<image>:<tag> docker images docker image ls docker pull <image> docker pull <image>:<tag> docker rmi <image> docker image rm <image> docker image prune docker image prune -a docker ps docker ps -a docker ps -q docker inspect <container> docker inspect <image> docker port <container> docker top <container> docker stats docker stats <container> docker run <image> docker run --name <name> <image> docker run -d <image> docker run -it <image> /bin/bash docker run -p <host_port>:<container_port> <image> docker run -v <host_path>:<container_path> <image> docker run --env KEY=VALUE <image> docker run --rm <image> docker run --network <network> <image> docker run --restart always <image> docker start <container> docker stop <container> docker restart <container> docker logs <container> docker logs -f <container> docker logs --tail 100 <container> docker exec -it <container> /bin/bash docker exec -it <container> sh docker exec <container> <command>	docker cp <container>:<path> <host_path> docker cp <host_path> <container>:<path> docker login docker logout docker push <image> docker push <repo>/<image>:<tag> docker pull <repo>/<image>:<tag> docker network ls docker network create <network> docker network inspect <network> docker network connect <network> <container> docker network disconnect <network> <container> docker network rm <network> docker network prune docker volume ls docker volume create <name> docker volume inspect <name> docker volume rm <name> docker volume prune docker system df docker system prune docker system prune -a docker system info docker compose version docker compose up docker compose up -d docker compose down docker compose down -v docker compose ps docker compose logs docker compose logs -f docker compose build docker compose pull docker compose push docker compose restart docker compose exec <service> <command> docker compose run <service> <command> docker compose config
---	--	--